

Master Prompt:

We are developing a personalized Virginia Tech student sidekick app. Our app's main focus is to assist students in finding events around the Virginia Tech campus based on initial questions and canvas availability.

Features:

Can add events to Canvas through API

Implementation Details:

Mobile app coded in React Native Expo

Backend coded in Node.js

Database in MongoDB Atlas

Different components should act as black boxes to each other:

Database offers functions for storing, retrieving data, etc. -> callers do not know how they are implemented

Agents return general form of data -> receivers do not know how data is retrieved

Databricks for data analytics, recommendations for events based on individual user data and past events

Interfaces:

Agents should output a common data shape. Agents have different ways of getting data, but they return information in the same way to make things easier for the consolidation agent and make development of the agents easier. The general data format should take this shape (still prone to changes depending on what data we want to work with):

```
{
  "type": "event" | "context"
  "data": string
  ...
}
```

Multiple agents:

4 Data Fetching Agents:

- GobblerConnect
- Canvas
- VT Sports
- Discord
- Google Calendar

1 Coordinating Agent:

- Consolidate information
- Remove duplicate events

1 User-Facing Agent:

- Give response to users

All agents should function through the Gemini API.

GobblerConnect agent:

Access public API

Canvas agent:

Access student's canvas through access token.

Access Calendar events, announcements, classes.

Add events to calendar

Canvas API: [Calendar Events | Instructure Developer Documentation Portal](#)

VT Sports Agent:

Access sports events via hokiesports API

<https://hokiesports.com/sports/football/schedule>

Discord Agent:

Pipeline:

1. Add discord bot to club discord
2. Select channels for announcements
3. Pass text from channels to agent
4. Process information then return through coordinating agent

Google Calendar:

Process Events

Add Events

Improvements:

Could be a UI improvement when adding busy/free times during first setup. Adding dates just appends text to the bottom which looks very ugly.

We should create a banner that shows live events when looking at the sports category. Sports entries seem bare right now, VT sports API should provide sufficient information:

- I see on hokie sports there is a link for a live stream, maybe we could embed this somehow?
- Opponent, Sport type, Date, Time, etc.
- Time might be inaccurate right now, I don't see a time for Virginia Tech v Chowderfest but on My Little Gobbler it says it ends at 11:59 EST
- Currently sport type is missing, which could lead users into thinking every sports event is football
- We also could pull up relevant news articles through a news API
- Some sport entries have TBD start/ending dates. We could ping periodically to fix that, or just put a disclaimer saying we might not have accurate dates.
- We could also pull up past records in a certain sport vs another university.

Data does appear to be broadly accurate to sources, but entries don't show much to the user. Our app should be a one-stop shop for all VT events, so we should try to put as much information as we can on one event. For example, instead of only showing the event in isolation, we can also show: past events, a summary of typical club activities, (potentially include social media posts?)

Adding to the calendar should not require start and end times like it currently does. It should suggest a general shape of calendar events, based on the current information. For example, if we only have the ending time, then only supply the end time of the event. Users should be able to modify this event if they want, so they aren't locked into a certain event if the details are wrong.

Instead of eliminating duplicate entries between sources, merge them together for more information. Because discord might have more information on the event than gobblerconnect.

We could do a periodic site wide evaluation of all the club groups through AI. This would mean asking AI to find any information about clubs like past events, purpose, members, etc. This might be really expensive so this would have to be run infrequently.

User-facing agent should be more personalized and informative. A user's agent should have memory of the user and their previous chats. The agent should have a site-wide memory of VT events like stated above. The user-facing agent does not speak to the user currently. It currently only has the purpose of filtering events. The agent should speak to the user and respond to queries, while also providing potential suggested events.

I like the schedule conflict notification, but there could be improvements in UI and information.

Prompt for black box implementation:

No back-end code should be present in front-end code, this is a hard stop rule. Front-end code should only call back-end interfaces, and act on data specified in the return of the interface. Front-end code should only influence front end UI, back-end code should be in functions which front-end code calls for a side effect or a data shape. I want you, from our specifications, to create a comprehensive list of backend functions and data types. The more we have defined, the easier it will be in the future, so define as much as possible. Make sure the side effects and

returns of functions show clearly what the function does. Make sure data types can be extended, but the old fields still remain there, so if we decide to add more data the frontend code does not break. I want you to define all of these interfaces within a file in a shared folder between backend and frontend. Additionally, backend itself should try to enforce this methodology as much as possible, but instead of backend to frontend it is scoped to specific backend functionality. I want you to describe and enforce this methodology to all future chats by adding an [agent.md](#) file while clearly describes what I have said.

Roles:

Redley - Discord bot:

- Manual fields through tagging
- Scoped to specific channels
- Maybe additional commands
- Other agents
- Canvas API
- Google Calendar API

John - Backend:

- MongoDB Atlas, implement database functions from defined interfaces
- Web hosting/cloud functions on vultr
- Maybe work on agents?
- Databricks analytics for user preferences

Arthur - UI/UX:

- Develop a general theme from inspirations
- Color scheme
- Define constraints and goals
- Make our logo
- First revamp landing page and login (add some flair make look less ai)
- Pop up errors/better looking errors

Ansh - Quality Testing

- UI feel and consistency
- Run unit/integration tests on all backend code
- Make sure backend and frontend work together
- Verification that something fulfills the purpose it was set out to do
- Put your agent on ultra mega conspiracy mode and test everything

Ansh please test:

- All contracts between frontend and backend
 - Do they both return and expect the same data?
 - Is the data itself correct for our use case?
 - Are we covering all the contracts?

- Is all code following our methodology of a black box between backend and frontend?

•

Knowledge:

4 agents with 4 different data sources: google calendar, canvas, gobbler connect, discord

Each one knows how to handle their specific platform but returns a common data shape.

One agent which acts as a consolidator between all of the agents which merges the data from all sources. One agent which acts as a user-facing assistant that has memory of the user and past events.

The data source agents (not google calendar or canvas because those are through personal access tokens) + consolidator run for the whole site on a recurrent schedule for base information about events. MongoDB holds data on all clubs: past events, members, (history of public interest maybe?). Agents match clubs + events with past club + events and concatenate to the memory. Canvas + google calendar + chat history are concatenated to user-specific memory.